

India's Health Transformation RAG Q&A System

Git-hub: <https://github.com/Vig1259/RAG-Q-A-Assistant-on-India-s-Health-Transformation>

Prepared for: ProAct AILabs

Project Report

A Retrieval-Augmented Generation system for question-answering over a single PIB background.

Source document	PIB backgrounder -"India's Health Transformation" (PRID=2269699)
Approach	Chunking → embedding's → cosine-similarity search → LLM answer generation (RAG)
Language / stack	Python, scikit-learn , Sentence-Transformers, Flask & Streamlit UI
LLM backends	Anthropic, OpenAI, Google Gemini, Groq - pluggable, with a no-key extractive fallback

1. Project Overview

This project answers natural-language questions about a single government press release the PIB backgrounder titled “India's Health Transformation” using Retrieval-Augmented Generation (RAG). Rather than asking a language model to answer from memory (which risks outdated or invented facts), the system first finds the most relevant passages from the actual document, then asks the model to answer using only those passages. This keeps every answer traceable back to a specific paragraph of the source document, and lets the system say “the document doesn't cover this” instead of guessing.

The pipeline has four stages, each implemented as its own module: ingest the document, split it into topical chunks, embed and index those chunks for similarity search, and retrieve + generate an answer. Two interfaces (a CLI and two optional web UIs) sit on top of the same underlying pipeline.

2. System Architecture

At a high level, a question flows through the system like this:

User question

- > embed question (same vector space as the indexed chunks)
- > cosine-similarity search over chunk embeddings (top-k)
- > build a prompt containing ONLY the retrieved chunks as context
- > LLM generates a short, grounded answer
- > UI shows the answer + which chunks/scores were used

The project is organized into small, single-responsibility modules:

- ingest.py - fetches and cleans the source document
- chunker.py - splits the cleaned text into 200–500 word topical chunks
- embeddings.py - pluggable embedding backend (TF-IDF or Sentence-Transformers)
- vector_store.py - builds, persists, and searches the embedding index
- rag.py - retrieval, prompt construction, and multi-backend answer generation
- cli.py / app.py / streamlit_app.py - three interchangeable interfaces over the same pipeline

3. Document Ingestion & Chunking

3.1 Ingestion

The PIB page is fetched over HTTP and parsed with BeautifulSoup. Headings, paragraphs, and list items are kept; images, scripts, the long outbound “References” link list, and the Hindi-language footer are discarded, since they add noise without adding answerable content. Bold section headers on the page (e.g. “Pillar 1: Public Health Insurance Through AB-PMJAY”) are converted into markdown-style `##` headings, which the chunker then uses as natural split points.

If the live page can't be reached no internet, or the request fails ingestion automatically falls back to a cleaned copy of the document that ships with the project. This means the whole pipeline can always be run end-to-end, even in an offline or sandboxed environment, without ever silently producing an empty index.

3.2 Chunking

The cleaned text is split at heading boundaries, since these already correspond to the topical sections of the backgrounder AB-PMJAY, Ayushman Arogya Mandirs, PM-ABHIM, ABDM, NHM maternal & child health programmes, TB/malaria/NCD elimination efforts, AI-in-healthcare initiatives, and AYUSH. Two rules keep chunks well-sized:

- Sections longer than 500 words are split further at paragraph boundaries (never mid-sentence) into ~350-word pieces.
- Sections shorter than 200 words are merged into a neighbouring section, but only if the merge doesn't push the result over 500 words this avoids creating either too-thin or too-bloated chunks.

The result is 15–16 chunks, each carrying a human-readable title, its word count, and its text the retrievable unit the rest of the system operates on.

4. Embedding's and Semantic Search

Embeddings are computed through a pluggable interface so the same codebase can run in two very different situations: completely offline with zero setup, or with genuine semantic understanding once network access and an extra package are available.

Backend	What it is	Setup
tfidf (default)	Scikit-learn TF-IDF vectors, 1–2 grams	None — runs fully offline, no download
sbert	all-MiniLM-L6-v2 dense sentence embeddings	pip install sentence-transformers (downloads model once)

Both backend implement the same `fit()/transform()` interface and produce L2-normalized vectors, so cosine similarity reduces to a simple dot product everywhere in the code — switching backend is a one-line environment-variable change and never touches the retrieval logic.

The index itself is a small set of flat files rather than a full vector database, since the corpus is only 15–16 chunks: an `embeddings.npy` matrix, a `chunks.json` metadata file, and a pickled/serialized copy of the fitted embedder so query-time vectors land in the exact same space as the indexed chunks. The search interface (`store.search(query, top_k)`) is intentionally the same shape a real vector database like

FAISS or Chroma would expose, so scaling up to a larger corpus later wouldn't require changing any calling code.

5. RAG: Retrieval, Prompting, and Answer Generation

For each question, the top-k most similar chunks are retrieved and inserted into a prompt as the model's only source of truth. The system prompt explicitly instructs the model to answer using only the given context and to say plainly when the document doesn't cover something, rather than filling gaps with outside knowledge.

5.1 Multiple interchangeable LLM backend

Rather than depending on a single provider, answer generation supports four LLM backend plus a fallback, tried in a configurable priority order:

- Anthropic API (Claude) - paid, requires billing setup
- OpenAI API (GPT models) - paid, requires billing setup
- Google Gemini - nominally free-tier, though some Google Cloud projects are provisioned with a locked 0 request/minute quota, which is an account-level issue rather than a code issue
- Groq - genuinely free, no credit card, no observed quota issues; recommended default for cost-free testing
- Extractive fallback - used automatically if no API key is configured or every backend call fails; ranks and returns the most relevant sentences from the retrieved chunks verbatim, guaranteeing a grounded, non-hallucinated answer with zero setup.

A user can force a specific backend either through an environment variable (LLM_BACKEND), a CLI flag (--backend groq), or, in the Streamlit interface, a live dropdown selector — so switching providers never requires editing code.

6. User Interfaces

Three interfaces sit on top of the identical rag.answer_question() call, so grounding behaviour is identical regardless of which one is used:

- CLI (cli.py) - interactive loop or one-shot question; prints the answer plus source chunk titles, snippets, and similarity scores
- Flask app (app.py) - a minimal single-page web form
- Streamlit app (streamlit_app.py) - a chat-style interface with a sidebar showing index status, a live backend selector, sample questions, and a green/red indicator confirming which backend actually answered the last question

7. Testing and Verification

Each stage was tested end-to-end against the real document content: chunk word-counts and titles were inspected directly, the TF-IDF index was built and queried with sample questions (e.g. “What are the four pillars of Ayushman Bharat?”, “What is PM-ABHIM's financial outlay?”), and both web UIs were smoke-tested via their respective test clients / syntax checks. The extractive fallback mode allowed full pipeline verification without requiring any paid API access.

8. Limitations and Future Improvements

- TF-IDF is lexical, not semantic - it can miss paraphrased questions that don't share vocabulary with the source chunk. Shipping sbert as the default would address this once model-download access is guaranteed.
- Extractive fallback answers are grounded but not fluent - they read as a list of quoted sentences rather than a synthesized answer. This is by design as a zero-cost safety net, not the intended primary experience.
- No re-ranking step - a cross-encoder re-ranker over the top-10 candidates would improve precision for a larger document.
- No automated evaluation set - a set of ~15 question/expected-chunk pairs would let retrieval accuracy be measured and regressions caught when tuning chunking or embedding's.
- Gemini free-tier quota reliability - during development, multiple freshly created Gemini API projects returned a hard 0 request/minute quota; this is a known, inconsistent Google-side provisioning issue rather than a defect in this codebase, and Groq was adopted as the practical free-tier default instead.

9. Setup & Run Instructions

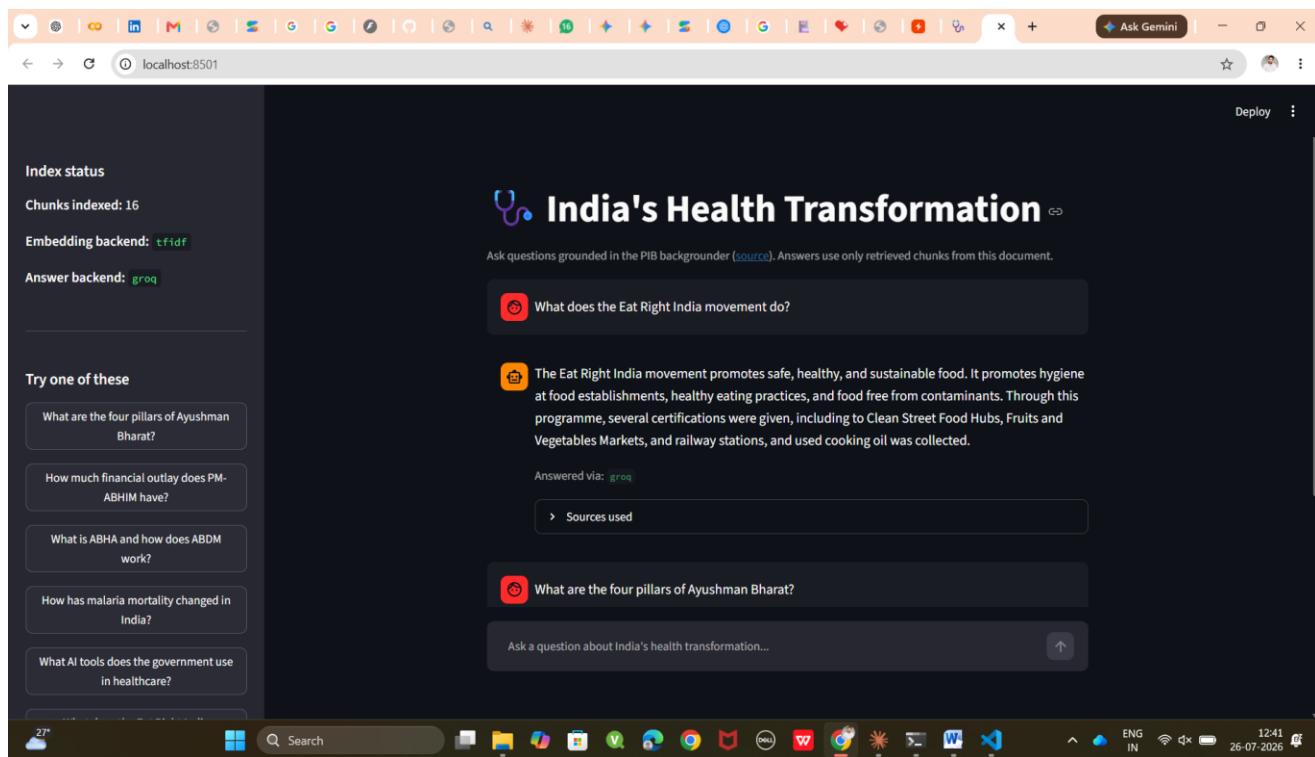
```
pip install -r requirements.txt
cp .env.example .env      # add one API key, e.g. GROQ_API_KEY
python -m health_rag.vector_store  # build the index
python run.py "What is AB-PMJAY?"  # CLI, one-shot
python -m streamlit run health_rag/streamlit_app.py  # Streamlit UI
```

Full setup details, environment variables, and troubleshooting notes are documented in README.md within the project repository.

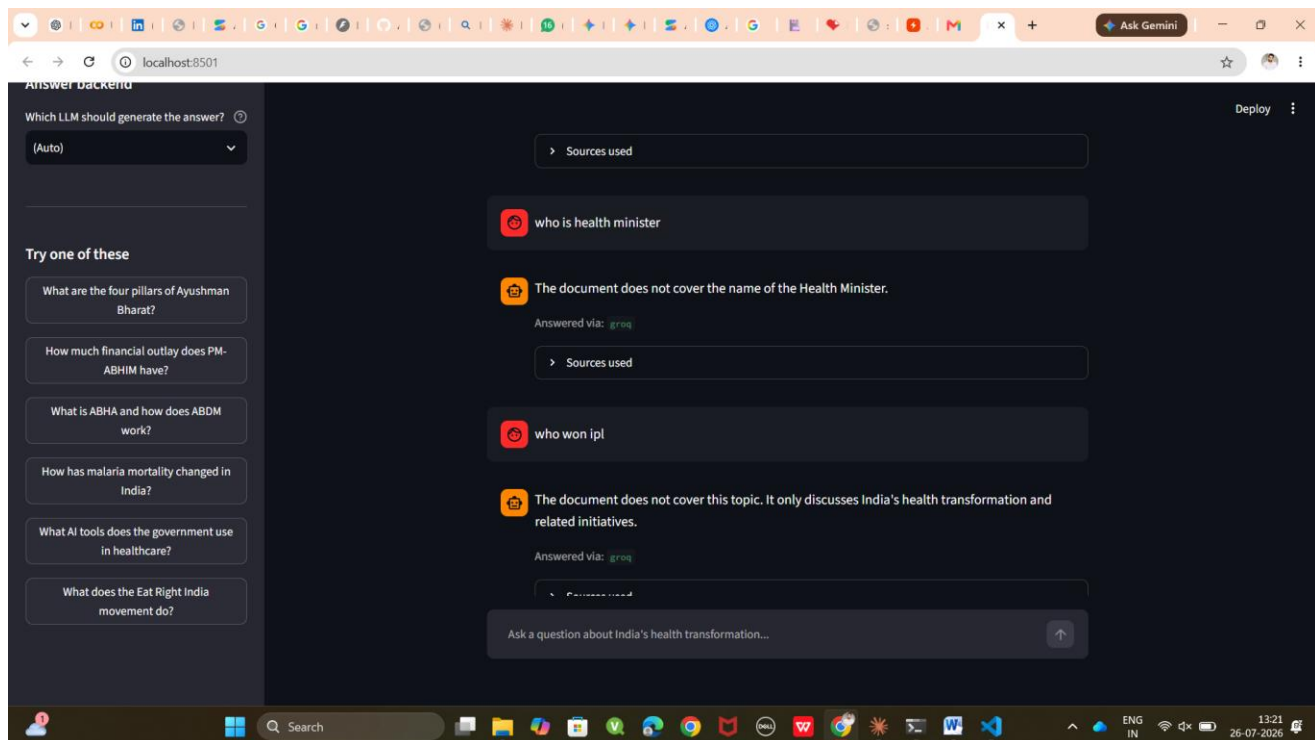
10. What I Had Learn / Research

- Studied the Retrieval-Augmented Generation (RAG) pipeline and how retrieval improves the accuracy of LLM-generated responses.
- Researched effective document chunking strategies (200–500 word chunks) to improve retrieval relevance while preserving context.
- Learned about semantic embeddings using Sentence-Transformers and compared them with TF-IDF as an offline fallback.
- Explored cosine similarity for ranking and retrieving the most relevant document chunks.
- Learned prompt engineering techniques to ensure the LLM generates answers only from the retrieved context, minimizing hallucinations.
- Researched vector indexing and embedding storage for efficient retrieval and reuse of embeddings.
- Explored integrating multiple LLM providers (OpenAI, Anthropic, Google Gemini, and Groq) with a common interface.
- Learned to implement fallback mechanisms to provide grounded extractive answers when an LLM or API is unavailable.
- Designed a modular RAG architecture with interchangeable ingestion, embedding, retrieval, and generation components for better scalability and maintainability.

11. User Interface



Queries on PIB document



Invalid queries not in document

12. References

- N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of EMNLP-IJCNLP, 2019.
- P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.